

CS571: Programming Languages

Assignment 1. Solution

Problem 1 [4pt] Write down the set of strings recognized by the following regular expressions. If the set is infinite, write down all strings that have 3 or fewer characters.

a) (2pt) $a|(ab|cde)(f|g)$

Solution $\{a, abf, abg, cdef, cdeg\}$

b) (2pt) $(aa|b)^*c^*$

Solution $\{\epsilon, b, c, aa, bb, bc, cc, aab, aac, baa, bbb, bbc, bcc, ccc, \dots\}$

Problem 2 [6pt]

a) (3pt) Build a regular expression that captures integers in the Racket programming language. For example:
123 -456 0123 #b1011 #o777 #xf12abcdef #b-10

The Racket integers must follow these rules:

1. The number can be base 10 (by default) or base 2 (starting with #b) or base 8 (starting with #o) or base 16 (starting with #x).
2. The number can have arbitrary number of legal digits (for its base). Base 16 numbers can have (a, b, c, d, e, f) as its digits.
3. The number can have an optional '-' but it must be after the base indicator such as #x if there is one.

Solution:

$(-?[0-9]+) | (\#b-?[01]+) | (\#o-?[0-7]+) | (\#x-?[0-9a-f]+)$

b) (3pt) Build a regular expression that captures all non-empty strings other than "abc". For your convenience, you may use the POSIX-extended regular expression $[\wedge]$ that takes a set of letters as argument and matches any other letter. For instance, $[\wedge abc]$ matches any (single) letter other than a, b or c. Use '.' to match any letter (wild cast).

Solution: Many. For example:

$. | .. | [\wedge a].. | a[\wedge b]. | ab[\wedge c] | \dots +$
 $[\wedge a].* | a | a[\wedge b].* | ab | ab[\wedge c].* | abc. +$
 $[\wedge a].* | a(\epsilon | [\wedge b].*) | ab(\epsilon | [\wedge c].* | c. +)$

Problem 3 [10pt] We have the following grammar with the start symbol E:

$E \rightarrow D \mid E + E \mid E - E$

$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

a) (2pt) Show a leftmost derivation for the expression "3 + 4 - 5"; show every step.

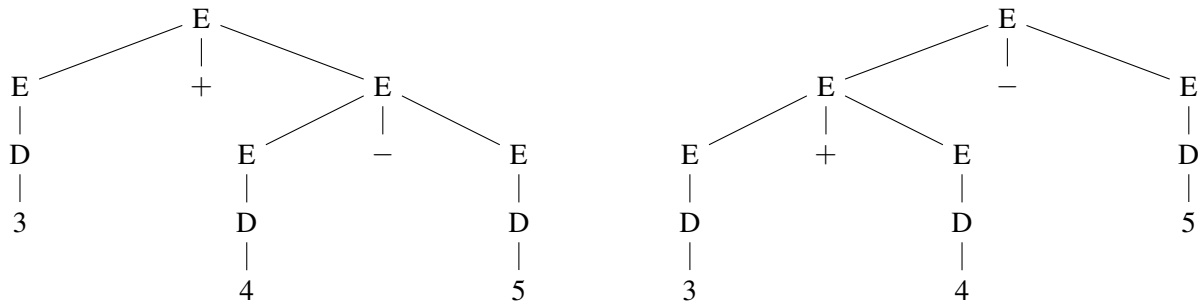
Solution: $E \Rightarrow E + E \Rightarrow D + E \Rightarrow 3 + E \Rightarrow 3 + E - E \Rightarrow 3 + D - E \Rightarrow 3 + 4 - E \Rightarrow 3 + 4 - D \Rightarrow 3 + 4 - 5$
or $E \Rightarrow E - E \Rightarrow E + E - E \Rightarrow D + E - E \Rightarrow 3 + E - E \Rightarrow 3 + D - E \Rightarrow 3 + 4 - E \Rightarrow 3 + 4 - D \Rightarrow 3 + 4 - 5$

b) (2pt) Show a rightmost derivation for the above expression; show every step.

Solution: $E \Rightarrow E - E \Rightarrow E - D \Rightarrow E - 5 \Rightarrow E + E - 5 \Rightarrow E + D - 5 \Rightarrow E + 4 - 5 \Rightarrow D + 4 - 5 \Rightarrow 3 + 4 - 5$
or $E \Rightarrow E + E \Rightarrow E + E - E \Rightarrow E + E - D \Rightarrow E + E - 5 \Rightarrow E + D - 5 \Rightarrow E + 4 - 5 \Rightarrow D + 4 - 5 \Rightarrow 3 + 4 - 5$

c) (2pt) Show two different parse trees for the above expression.

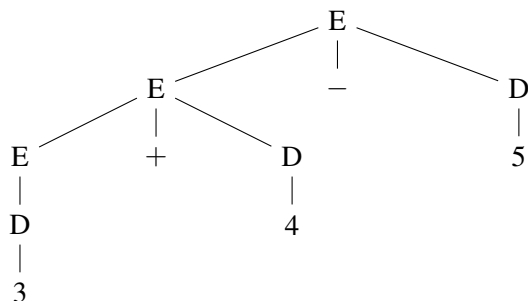
Solution:



d) (2pt) The grammar is ambiguous. Show a new grammar that removes the ambiguity and makes “+” and “−” left-associative. Show the parse tree for “3 + 4 − 5” in your new grammar.

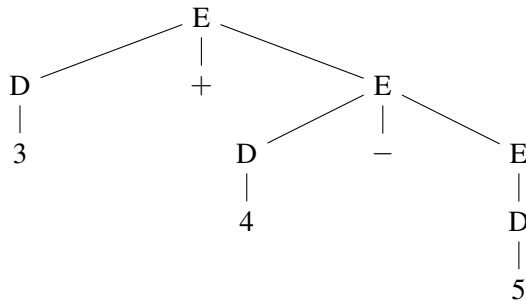
Solution: change first line to: $E \rightarrow D \mid E + D \mid E - D$

This is the only parse tree because for the first step of derivation, the only production rule we can use is $E \rightarrow E - D$ because if we use $E \rightarrow D$ or $E \rightarrow E + D$, we will not be able to obtain the “−5” no matter what we do later. So the first level from the tree root is $E - D$, and then the left E can only be using the production rule $E \rightarrow E + D$ to produce $3 + 4$ in the expression. So every step of the parsing tree is produced with the only possible production rule. There is no other parse tree for this expression in the new grammar.



e) (2pt) Show a new grammar that removes the ambiguity and makes “+” and “−” right-associative. Show the parse tree for “3 + 4 − 5” in the new grammar.

Solution: change first line to: $E \rightarrow D \mid D + E \mid D - E$



Problem 4 [6pt] Consider the context free grammar:

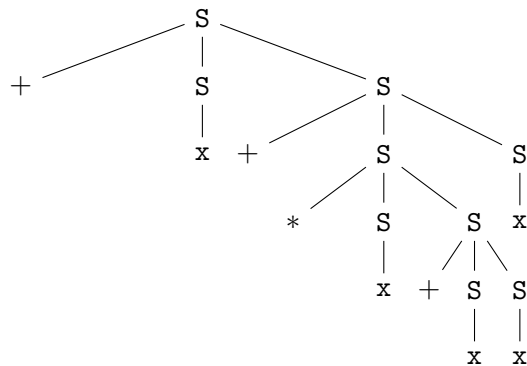
$$S \rightarrow +SS \mid *SS \mid x$$

where x is a terminal for letter x , and a string “ $+x + *x + xxx$ ”.

a) (2pt) Write down the rightmost derivation for the string.

Solution: $S \Rightarrow +SS \Rightarrow +S + SS \Rightarrow +S + Sx \Rightarrow +S + *SSx \Rightarrow +S + *S + SSx$
 $\Rightarrow +S + *S + Sxx \Rightarrow +S + *S + xxx \Rightarrow +S + *x + xxx \Rightarrow +x + *x + xxx$

b) (2pt) Draw the *concrete* parse tree for the string.



Solution:

c) (2pt) Is this grammar ambiguous or unambiguous? If it is ambiguous, give an equivalent unambiguous grammar.

It is unambiguous. For every step of the derivation, the production we can use is uniquely determined by the prefix operator. If the part of the string we are parsing starts with a ‘+’ we use $S \rightarrow + S S$, if it starts with a ‘*’ we use $S \rightarrow * S S$, or if it is an ‘x’ we use x . So the parse tree for a prefix expression is unique and the grammar is unambiguous.

Problem 5 [4pt] For each of the following grammars state whether it is ambiguous or unambiguous. If it is ambiguous, give an equivalent unambiguous grammar and state all the precedence and the associativity enforced by the unambiguous grammar. If it is unambiguous, state all the precedence and the associativity enforced by the unambiguous grammar.

a) Start symbol is E .

$$\begin{aligned}
 E &\rightarrow E + F \mid F \\
 F &\rightarrow F * F \mid D \mid (E) \\
 D &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
 \end{aligned}$$

Solution: ambiguous. New grammar:

$$\begin{aligned}
 E &\rightarrow E + F \mid F \\
 F &\rightarrow F * G \mid G \\
 G &\rightarrow D \mid (E) \\
 D &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
 \end{aligned}$$

In this new grammar, $*$ has higher precedence than $+$; both $+$ and $*$ are left associative.

b) Start symbol is E.

$$\begin{aligned}
 E &\rightarrow F * E \mid F \div E \mid F \\
 F &\rightarrow F + G \mid F - G \mid G \\
 G &\rightarrow D \mid (E) \\
 D &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
 \end{aligned}$$

Solution: unambiguous. $*$, $\div$ are right-associative, $+$, $-$ are left-associative. $+$ and $-$ have higher precedence than $*$ and $\div$.